

■ Programmation orienté objet

- Qu'est-ce qu'un objet?
 - ✓ Variable d'un type spécifique appelé Classe
 - ✓ Ou un représentant (instance) d'une classe
 - ✓ Par exemple la BMW 320i est un objet de la classe Voiture
- Qu'est-ce qu'une classe?
 - ✓ Type spécial (comme une structure) qui regroupe:
 - Des données ou variables
 - Des traitements (fonctions ou procédures) appelés méthodes
 - Et d'autres éléments
 - ✓ Par exemple la classe voiture contient des variables telles que roues, portières et des méthodes : démarrer, rouler

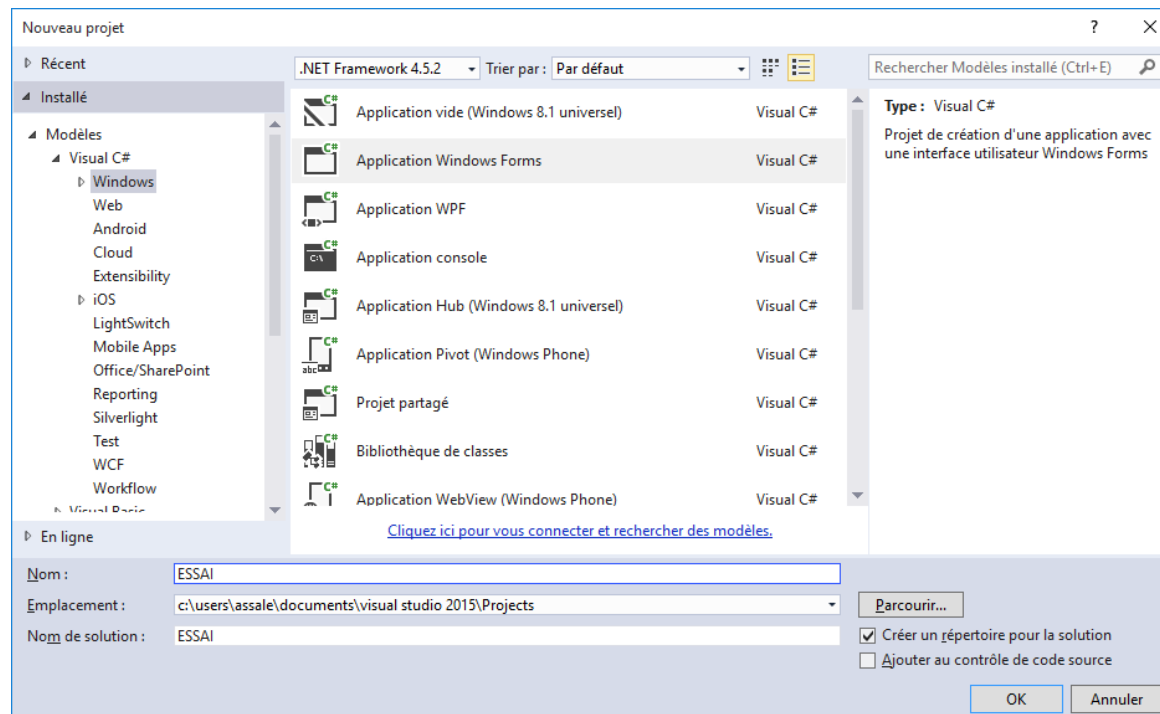
■ Visual Studio

- Environnement de développement intégré orienté objet, visuel (objets visibles) et événementiel
- Regroupe plusieurs langages dont VB - C# - C++ ...
- Dispose d'un système de types communs (CTS)
- Chaque langage définit les alias pour chaque type du CTS
- Projet base de développement est:
 - ✓ d'un modèle (application windows, web, ...)
 - ✓ composé plusieurs fichiers

Généralités

■ Visual Studio

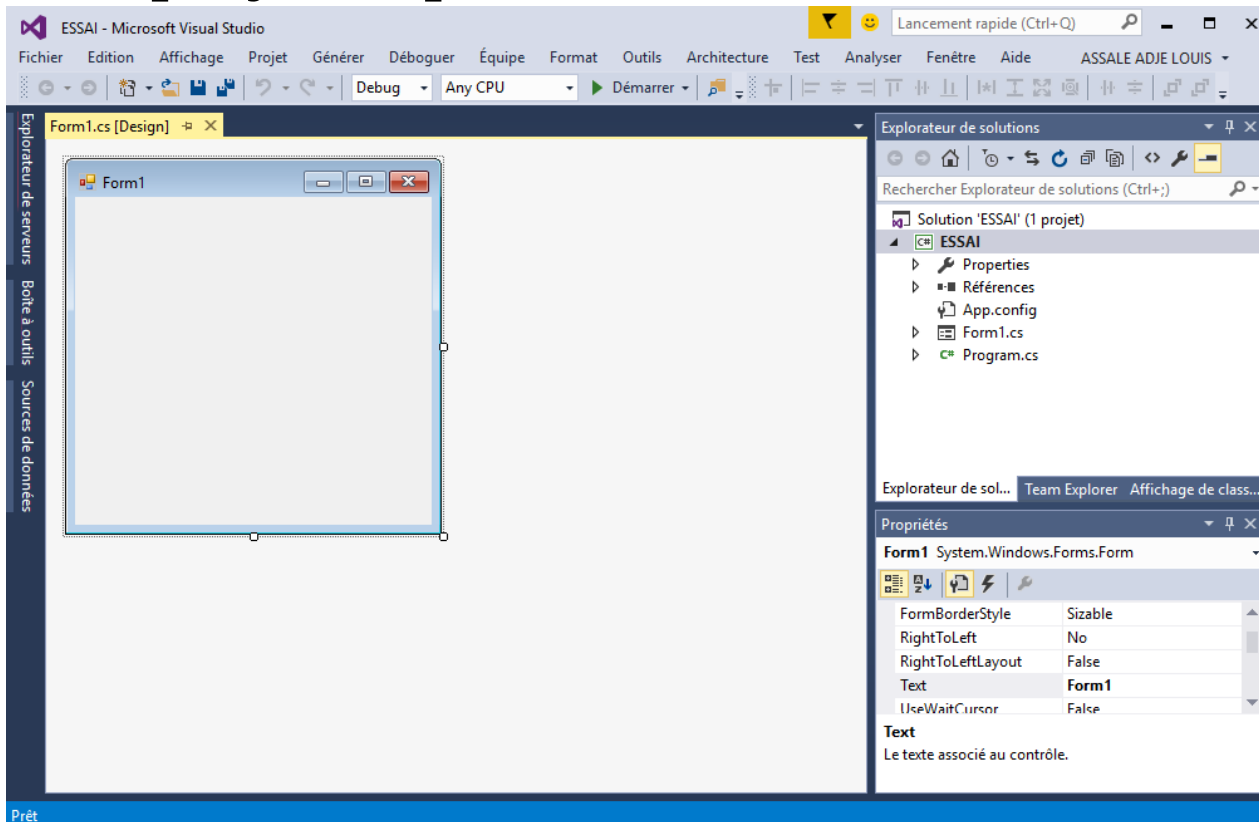
- Lancer Visual Studio et créer un nouveau projet de type Application Windows Form nommer le Essai



Généralités

■ Visual Studio

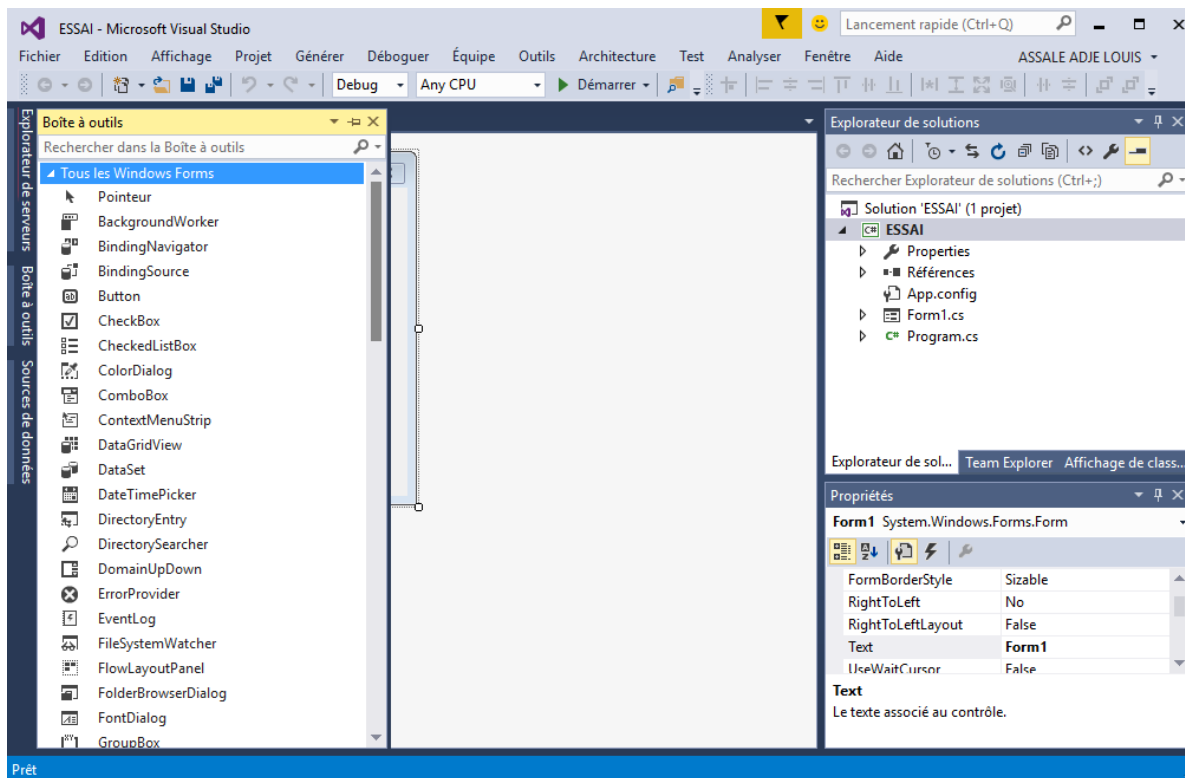
- Le projet se présente comme suit:



Généralités

■ Visual Studio

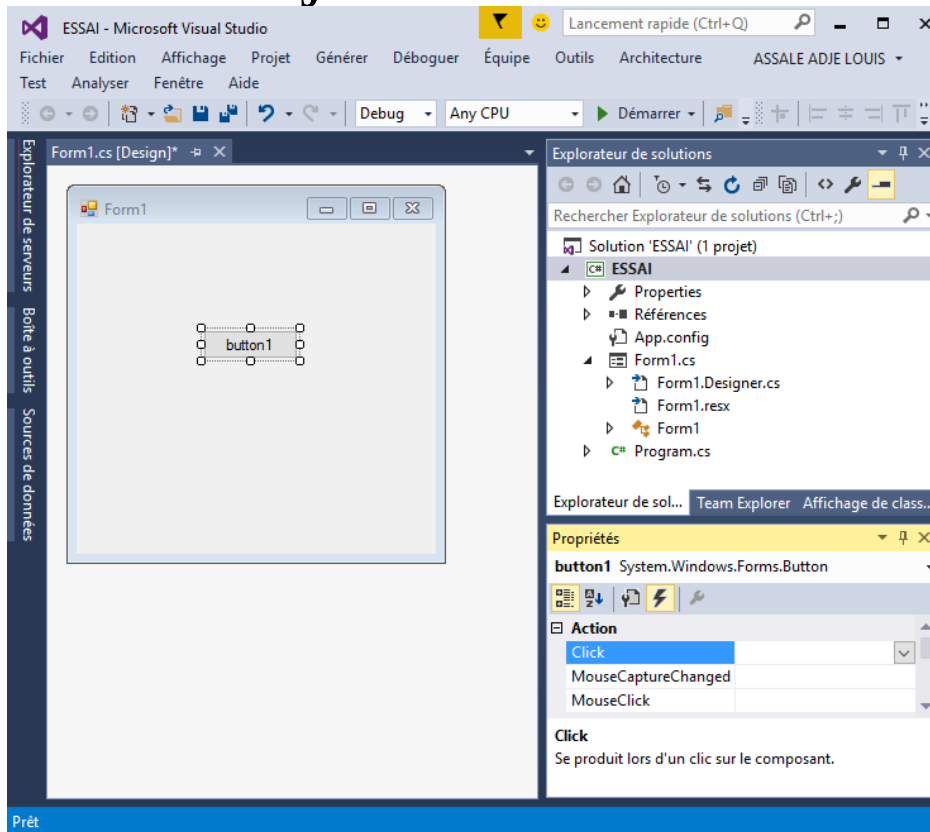
- On clique sur Boite à outils pour faire apparaître les objets visuels



Généralités

■ Visual Studio

- Déposer un objet sur la feuille



■ Visual Studio

- Développer en Visual Studio consiste à :
 - ✓ Dessiner l'interface :
 - On dépose sur une feuille les objets nécessaires
 - On les agence selon notre goût
 - ✓ Animer l'interface réalisée en écrivant du code, de ce fait on se pose 2 questions :
 - Où écrire le code : dans quel gestionnaire d'événement associé à quel objet?
 - Quoi écrire? C'est-à-dire le programme, cela nécessite une connaissance du langage à utiliser

■ Le langage C#

- Les types : alias des types de base de Visual Studio (CTS)
 - ✓ correspondance entre chq type du CTS et l'alias en C#, ainsi
 - System.Object donne **object**, System.String → **string**, System.Int32 → **int**, System.Byte → **byte**, System.Int64 → **long**, System.Char → **char**, System.Single → **float**, System.Double → **double** et System.Boolean → **bool** pour ne citer que ceux-ci
 - ✓ Conversion entre types
 - la conversion entre les types simples char, int et float est implicite
 - Conversion entre une chaîne et les autres types est définie par :
 - nombre → chaîne **nombre.ToString()**
 - chaîne → int ou long ou float ou double **{int|long|float|double}.Parse(chaine)**

■ Le langage C#

- Déclaration de variables
 - ✓ Syntaxe : **<type> var1, var2, ...;**
 - Exemple1 : **int i, j, k;**
 - Exemple2 : **const double PI = 3.14;** déclaration de constante
 - ✓ Tableaux
 - **<type> [] nomtableau** : tableau à une dimension
 - **<type> [,...] nomtableau** : tableau à plusieurs dimensions
 - Création du tableau à l'aide du mot clé new : **nomtableau = new type [taille]**
 - Exemple : **int [,] entiers; entiers = new int[2,3,4];**
 - Tableaux imbriqués ou irréguliers : **<type> [][] nomtableau**

■ Le langage C#

- Déclaration de variables structures et énumérations

- ✓ Structure : *struct* **<type> var1, var2, ... {**
 <type1> <liste_de_champs>;
 <type2><liste_de_champs>;
 ...
 }

- ✓ Énumération : *enum* **<type> var1, var2, ... {**
 valeur1,
 valeur2,
 ...
 }

■ Le langage C#

- Opérateurs et expressions
 - ✓ Opérateur d'affectation =
 - Affectation simple : `i = 2;`
 - Raccourcis d'affectation `op=` : `i=i/2;` $\Leftrightarrow$ `i/=2;`
 - ✓ Incrémentation/décrémentation : `i++;` `++i;` `--i;` `i--;`
 - ✓ Opérateurs logiques :
 - `==` : égalité `if(x == 2)`
 - `<=`, `<`, `=>`, `>` : opérateurs de comparaison
 - `!`, `&&`, `||` : non, et, ou Ex (`x!=y`)
 - ✓ Opérateurs sur les bits :
 - `~`, `&`, `|`, `^`, `<<`, `>>` : non, et, ou, ou exclusif, décalage à gauche et droit

■ Le langage C#

- Opérateurs et expressions
 - ✓ Autres type d'expressions
 - Conditionnelle : `?:` Exemple `y = (x>0)?x-2:x+3;`
 - Taille d'une variable ou type : `size` Exemple `sizeof(int)` ou `sizeof(x)`
 - ✓ Expressions simples et composées
 - Expression simple se termine par un point virgule (;), Ex : `i++; j=k-2;`
 - Expression composée commence par une accolade ouvrante et se termine par une accolade fermante, peut contenir des déclarations, Ex :

```
{ int aux;  
  aux = i;  
  i = j;  
  j = aux;  
}
```

■ Le langage C#

- Les instructions

- ✓ Sélection **if**

- syntaxe : ***if*** (*<expression>*)
 <instruction>
 [*else* *<instruction>*]

- Exemple : ***if*** (*i>0*) *i++*; *else i++*;

- ✓ Sélection **switch**

- Syntaxe : **switch** (*<expression>*)
 { *case* *<expression constante>*: *<liste d'instructions>*
 ...
 [*default* : *<liste d'instructions>*]
 }

■ Le langage C#

- Les instructions

- ✓ Sélection **switch**

- Exemple : *char c*;

```
switch (c)  
{ case 'd' :  
    case 'c' : supprimer(); break;  
    case 'i' : inserer(); break;  
    case 'm' : modifier ( );break;  
    case 's' : sortir(); break;  
    default : erreur();  
}
```

■ Le langage C#

- Les instructions

- ✓ répétition **while**

- Syntaxe : **while** (<expression>) **corps** ;

- Exemple : **while** (*i*<40) {
 j = j+2;
 i++; }

- ✓ Répétition **do/while**

- Syntaxe : **do**
 corps
 while (<expression>);

- Exemple : **do** *i--;* **while**(*i*);

■ Le langage C#

- Les instructions
 - ✓ répétition **for**
 - Syntaxe : *for (initialisation ; expression ; pas) corps*
 - Exemple : *for (i=0; i<n;i++) j=j*2;*
 - ✓ Répétition **foreach**
 - Syntaxe : *for (variable in expression) corps*
 - Exemple :
 - ✓ De débranchements
 - **break** : permet de sortir d'une répétition ou conditionnelle
 - **continue** : permet de passer à l'itération suivante dans une répétition
 - **return** expression : spécifie la valeur de retour

■ Le langage C#

- Les procédures et fonctions
 - ✓ Syntaxe : **<type> <nom_ss_programme>(liste_parametres)**
corps
 - ✓ Pour une procédure le type est nécessairement **void** et le corps ne contient pas l'instruction **return**
 - ✓ Exemple de fonction qui calcule la puissance n d'un réel :
 - ***double power (int n, double x) {***
double p = 1.0;
while(n--) ***p *= x;***
return (p);
}

■ Le langage C#

- Notion de classe

- ✓ Syntaxe de déclaration :

```
■ <modificateur> class <Nomclasse> {  
    <modificateur> <type> <nom_membre_simple> ;  
    ...  
    <modificateur> <type> <nom_méthode>(<liste paramètres>  
        { corps méthode} ;  
    ...  
}
```

- ✓ Membres d'une classe :

- Champs : variable contenant des valeurs les données de l'objet
- Méthode : procédure ou fonction opérant sur les champs
- Propriété : méthode qui donne l'impression d'être un champ

■ Le langage C#

- Notion de classe
 - ✓ Membres d'une classe :
 - Constante : champ contenant une valeur non modifiable
 - Indexateur : membre permettant d'indexer un objet via des méthodes **Get** et **Set**
 - Événement : action, par exemple clic sur le bouton de la souris
 - Opérateur : en C#, l'on peut surcharger des opérateurs
 - ✓ Modificateurs applicables aux membres :
 - **public** : membre accessible de l'extérieur (de partout)
 - **protected** : membre inaccessible de l'extérieur, exceptés les classes dérivées
 - **private** : accessible qu'aux membre de la même classe, valeur par défaut

■ Le langage C#

- Notion de classe
 - ✓ Modificateurs applicables aux classes :
 - **public** : classe visible par n'importe quel programme d'un autre espace de nom
 - **protected** : classe visible seulement par toutes les autres classes héritant de la classe conteneur de cette classe
 - **private** : classe visible uniquement par toutes les autres classes du même espace de nom où elle est définie
 - **internal** : classe visible uniquement par toutes les autres classes du même *assembly*
 - **abstract** : classe non instanciable, aucun objet ne peut être créé, ce modificateur peut être associé à **public**, **protected** ou **internal**
 - Sans mot clé : classe qualifiée de **internal** ou **private** si classe interne

■ Le langage C#

- Notion de classe
 - ✓ Méthodes d'une classe :
 - C'est des fonctions ou procédures définissant un code opérant sur les données de l'objet
 - Méthode **Main** : elle est unique, de statut publique et statique, elle sert de point d'entrée dans l'application
 - Constructeurs : méthodes spéciales qui portent le même nom que la classe, ne retournent pas de valeur et permettent d'initialiser les données de l'objet (création d'une instance)
 - `<classe> <nom_objet> = new <classe>(arguments du constructeur)`
 - Membre statique (mot clé **static**) : définit le membre au niveau global et non au niveau d'une instance; l'accès est **nomClasse.nomMembre**

■ Le langage C#

- Notion de classe
 - ✓ Méthodes et classes abstraites :
 - On utilise le mot clé **abstract** pour définir une telle méthode ou classe
 - Une méthode abstraite n'a pas de corps donc pas exécutable doit être obligatoirement redéfini dans une classe fille
 - Si une classe qui contient au moins une méthode abstraite, elle doit être déclarée abstraite. Une classe abstraite peut ne pas contenir du tout de méthodes abstraites
 - Une classe abstraite n'est pas instanciable, elle doit être dérivée
 - Une classe dérivée qui ne redéfinit pas au moins une méthode abstraite de la classe mère ou qui contient une méthode abstraite doit être déclarée abstraite

■ Le langage C#

- Notion d'interface
 - ✓ Syntaxe de déclaration

```
interface nomInterface {  
    //déclaration de méthode  
    type Methode1() ;  
    //déclaration de propriété  
    type Propriete { get ;}  
    //déclaration d'événement  
    event typeEvenement Evenement ;  
    //déclaration d'indexateur  
    type this[int indx] {get ; set ;}  
}
```

■ Le langage C#

- Notion d'interface
 - ✓ C'est un contrat qui peut contenir que des :
 - propriétés, méthodes, indexateurs ou événements, pas de champ ni d'attribut
 - Ne contient pas de méthodes déjà implémentées, rien que les signatures des propriétés et méthodes
 - Tous les membres d'une interface sont automatiquement public
 - On peut construire une hiérarchie d'interfaces
 - Pour construire un objet à partir d'une interface, on crée une classe non abstraite qui implémente tous les membres de l'interface.

■ Le langage C#

- Notion d'héritage
 - ✓ Consiste à définir une classe en s'appuyant sur une autre classe
 - ✓ Syntaxe : ***class ClasseDérivée : ClasseBase { ... }***
dans le corps de la classe dérivée, on peut ajouter des membres comme modifier des membres
 - ✓ C# ne gère pas l'héritage multiple de classe, mais autorise l'héritage multiple d'interface :
class TotoTataTiti : Toto, Itata, Ititi {
...
} Toto est une classe, Itata et Ititi des interfaces
 - ✓ Une classe scellée (modificateur ***sealed***) ne peut être dérivée

■ Application console

- Définition
 - ✓ **Application** qui s'exécute en environnement ligne de commandes DOS
- Méthodes d'entrées/sorties de la classe **Console**
 - ✓ **Console.Read()** : lit un caractère
 - ✓ **Console.ReadLine()** : lit une chaîne de caractères
 - ✓ **Console.Write()** : écrit le contenu d'une variable ou message
 - ✓ **Console.WriteLine()** : écrit et retourne à la ligne

■ Objets usuels

- Propriétés communes aux objets d'une feuille
 - ✓ **Name** : nom de l'objet doit être unique sur une feuille; permet l'accès aux propriétés et méthodes par : ***nom_objet.propriété***
 - ✓ **TabIndex** : rang d'accessibilité de l'objet sur une feuille en appuyant sur la touche tabulation
 - ✓ **Text** : titre apparaissant dans l'objet
 - ✓ **Enabled** : à True l'objet est actif sur la feuille à l'exécution
 - ✓ **Visible** : à False l'objet n'est pas visible à l'exécution

■ Objets ou contrôles usuels

- Objet de type étiquette : **Label**
 - ✓ Propriété importante est : **Text** permet affichage d'information
 - ✓ Événement important associé :
 - **TextChanged** : le code qui y est défini est exécuté quand la valeur de la propriété Text change
- Objet de type zone de saisie : **TextBox**
 - ✓ Propriété importante est : **Text** qui affiche et récupère le texte saisi dans la zone

■ Objets ou contrôles usuels

- Objet de type zone de saisie : **TextBox**
 - ✓ Événements importants associés:
 - **TextChanged** : le code qui y est défini s'exécute quand le contenu de la zone de saisie change
 - **KeyPress – KeyUp - KeyDown** : le code qui y est défini s'exécute quand on appuie sur une touche du clavier – utilisé pour le contrôle de saisie de caractères dans le TextBox
 - **GotFocus – LostFocus** : le code est exécuté lorsque le focus (la main) est respectivement donnée au TextBox ou perdue par le TextBox

■ Objets ou contrôles usuels

- Objets de type liste : **ListBox** et **ComboBox**
 - ✓ Propriétés importantes :
 - **Text** : renvoie l'élément courant de la liste
 - **SelectedIndex** : renvoie ou définit l'index de l'élément sélectionné
 - **SelectedText** : pour un ComboBox renvoie ou définit le texte de l'élément courant
 - **SelectionMode** : pour un ListBox, valeur à **One** permet la sélection d'un seul élément – à **MultiSimple** la sélection de plusieurs éléments consécutifs – à **MultiExtended** la sélection de plusieurs éléments discontinus
 - **Items** : permet en mode conception de saisir les valeurs des éléments de la liste

■ Objets ou contrôles usuels

- Objet de type liste : **ListBox** et **ComboBox**
 - ✓ Méthodes de la collection Items :
 - **Add** : permet d'ajouter un élément en fin de liste. Ex *ListBox1.Items.Add("toto")*
 - **Insert** : permet d'ajouter un élément à une position spécifique. Ex *ListBox1.Items.Insert("toto« , i)*
 - ✓ Événement important :
 - **SelectedIndexChanged** : le code qui y est inscrit est exécuté quand on sélectionne un élément

■ Objets ou contrôles usuels

- Objets de type Choix : **CheckBox** et **RadioButton**
 - ✓ Propriétés importantes :
 - **Text** : définit le titre de l'élément
 - **Checked** : valeur à **True**, signifie que le contrôle a été choisi
 - Pour les **RadioButton** (bouton d'option), il y en a au moins 2 et l'on ne peut choisir qu'un seul dans le lot
 - ✓ Événement important :
 - **CheckedChanged** : le code qui y est inscrit est exécuté quand on choisit l'élément

■ Objets ou contrôles usuels

- Objet de type Bouton de commande : **Button**
 - ✓ Propriétés importantes :
 - **Text** : définit le titre du bouton
 - **Image** : définit l'image à afficher sur le bouton
 - ✓ Événement important :
 - **Click** : le code qui y est inscrit est exécuté quand l'on clique sur le bouton

■ Objets ou contrôles usuels

- Paramètres des gestionnaires d'événements
 - ✓ Un gestionnaire d'événements est une méthode liée à un événement (comme touche appuyée, clic de souris, ...)
 - ✓ Les gestionnaires d'événements présentent 2 types de paramètres:
 - **Object** : le premier paramètre est de type object
 - **EventArgs** / **xxxEventArgs** : le 2nd paramètre, selon le gestionnaire est de type EventArgs ou xxxEventArgs (comme KeyPressEventArgs pour l'appui sur une touche dans une zone de saisie, MouseEventArgs pour le clic de souris)

■ Objets ou contrôles usuels

- Paramètres des gestionnaires d'événements
 - ✓ Le paramètre *e* de type ***KeyPressEventArgs***
 - Il sert à contrôler la saisie de chaque touche (caractère) dans une zone de texte
 - Il dispose de la propriété ***KeyChar*** qui renvoie le caractère appuyé dans la zone de saisie
 - Cette propriété (***KeyChar***) sert aussi à afficher dans la zone le caractère qui lui est affecté dans le gestionnaire d'événement

■ Objets ou contrôles usuels

- Paramètres des gestionnaires d'événements
 - ✓ Le paramètre *e* de type *MouseEventArgs*
 - Il sert à gérer les appuis sur les boutons de la souris
 - Il dispose des propriétés :
 - **Button** : indique quel bouton a été appuyé
 - **Clicks** : le nombre de clics sur le bouton
 - **X | Y** : coordonné dans l'axe des X ou des Y d'un clic de souris
 - **Delta** : nombre de crans lors de la roulette de la souris

■ Objets ou contrôles usuels

- L'objet de type boîte de message : **MessageBox**
 - ✓ Dispose de la méthode **Show()** qui affiche une boîte de message
 - `MessageBox.Show(texte, titre, lesboutons, licône, leboutonParDéfaut)`
 - `lesboutons` : boutons qui seront affichés; les valeurs possibles sont : `MessageBoxButtons.{AbortRetryIgnore | Ok | OkCancel | RetryCancel | YesNO | YesNoCancel}`.
 - `Licône` : l'icône à afficher; les valeurs possibles sont : `MessageBoxIcon.{Asterisk | Error | Hand | Information | None | Exclamation | Question | Stop | Warning}`.
 - `leboutonParDéfaut` : bouton sélectionné par défaut; valeur possibles: `MessageBoxDefaultButton.{Button1 | Button2 | Button3}`
 - Ex : **`MessageBox.Show("la valeur doit être non nulle", "Erreur", MessageBoxButtons.OkCancel, MessageBoxIcon.Stop)`**

Visual Studio

■ Travaux pratiques N°1

- Soit à réaliser cette application
 1. Dessiner l'interface; changer la propriétés Name des objets suivants
textBox1 en Entier1 – textBox2 en Entier2 – radioButton1 en pgcd – radioButton2 en produit – radioButton3 en reste – valeur en resultat – button1 en Calculer – button2 en Nouveau et button3 en Quitter
 2. Écrire le code de sorte que si des valeurs sont saisies et une option choisie, à l'appui sur le bouton Calculer la valeur s'affiche dans le label resultat

Calcul

Entrer 2 entiers

Entier1 Entier2

☐ Calculer le pgcd ☐ Calculer le produit ☐ Calculer le reste de la division

Résultat

Valeur

Calculer Nouveau Quitter

■ Travaux pratiques N°1

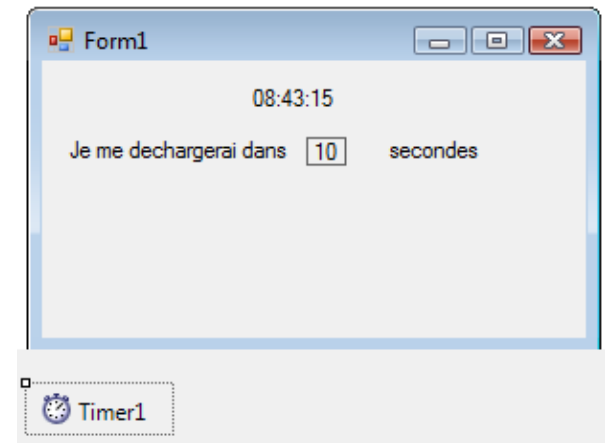
- suite
- 3. Écrire du code de sorte que quand l'on clique sur nouveau les valeurs s'effacent et le curseur est positionné dans Entier1 – écrire le code pour le bouton Quitter
- 4. Écrire du code de sorte que Calculer ne s'active que quand il y a des valeurs dans les zones de saisie entier1 et entier2
- 5. Écrire du code de sorte que dans entier1 et entier2 on n'autorise que les touches 0 à 9, ← (Backspace) et ↵ (Enter)
- 6. Écrire du code pour afficher un message en cas de saisie de 0 dans entier2 pour les options pgcd et reste de la division

■ Autres types d'objets

- L'objet de type minuterie : **Timer**
 - ✓ Définit une minuterie qui dispose de la propriété ***Interval*** :
 - Cette propriété est un nombre exprimé en millisecondes
 - À chaque intervalle de temps de la valeur de la propriété ***Interval***, le code inscrit dans le gestionnaire d'événements ***Tick*** de l'objet ***Timer*** est exécuté
- L'objet de type date : **DateTime**
 - ✓ Définit une date complète
 - La propriété ***Now*** renvoie la date courante y compris l'heure
 - On convertit en chaîne de caractères cette date en utilisant un format :
 - Le format "***hh:mm:ss***" ou "***HH:mm:ss***" renvoie l'heure courante au format 12 ou 24 heures

■ Travaux pratiques N°2

- Soit l'application suivante
 1. Afficher l'heure courante à l'aide du contrôle **Timer** et d'un contrôle **Label**.
 2. Programmer le contrôle Timer de sorte que la feuille se décharge au bout d'un certain nombre de secondes par affichage d'un compte à rebours dont la valeur initiale est 10.



■ Autres types d'objets

- L'objet de type Image : **PictureBox**
 - ✓ Sert à afficher une image au format : JPEG – GIF – BMP ...
 - ✓ Utilise la propriété **Image** pour afficher une image
 - En mode conception : on renseigne cette propriété en double-cliquant sur les trois points pour ouvrir une fenêtre de choix d'image
 - En mode programmation : on renseigne cette propriété à l'aide de la méthode **FromFile** de l'objet **Image** :
PictureBox1.Image = Image.FromFile(nom_fichier_image)
 - ✓ La propriété **SizeMode** définit le placement et le dimensionnement de l'image dans le **PictureBox**
 - ✓ On affecte l'image d'un PictureBox à un autre par :
PictureBox1.Image=PictureBox2.Image

■ Autres types d'objets

- L'objet de type Image : **ImageList**
 - ✓ Enregistre une collection d'images non visible à l'utilisateur
 - ✓ Ajout d'images via la propriété collection Images:
 - En mode conception : on renseigne cette propriété en double-cliquant sur les trois points pour ouvrir une fenêtre de choix d'image
 - En mode programmation : on utilise la méthode **Add** de cette propriété de la manière suivante : `ImageList1.Images.Add (mon_image)`, avec `mon_image` un objet Image récupéré d'un fichier à l'aide de la méthode ***FromFile(nom_fichier_image)***
 - ✓ Récupération d'images de ImageList via la collection Images :
 - `PictureBox1.Image = ImageList1.Images[i]` i rang de l'image à récupérer dans la collection

■ Autres types d'objets

- L'objet de type Image : **ImageList**
 - ✓ Suppression d'images de la collection à l'aide de méthodes :
 - `remove(i)` : supprime l'élément de rang `i`
 - `clear()` : supprime toutes les images de la collection
 - ✓ La collection d'images est exploitée par les contrôles disposant de la propriété `ImageList` à savoir : `Button` – `CheckBox` – `Label` – `ListView` – `RadioButton` – `TabControl` – `ToolBar` – `TreeView`
 - Il suffit d'affecter à la propriété `ImageList` du contrôle en question le nom d'un objet `ImageList` pour lier la collection d'images à ce contrôle

■ Autres types d'objets

- Objet de type Calendrier : **DateTimePicker**
 - ✓ permet la sélection d'un élément parmi une liste de dates et heures
 - ✓ La propriété **Value** permet de récupérer la date sélectionnée
 - ✓ Le gestionnaire d'événements **ValueChanged** permet de définir ce qu'il faut faire en cas de sélection d'une date
 - ✓ Exemple: la date sélectionnée dans le ***DateTimePicker1*** est affectée à un ***TextBox1***, on écrit ceci dans l'événement ***ValueChanged***
 - ***TextBox1.Text = DateTimePicker1.Value.ToShortDateString();***

■ Autres types d'objets

- Autres listes : **DomainUpDown** - **NumericUpDown**
 - ✓ DomainUpDown = zone de texte avec paire de boutons (flèches)
 - Propriétés **Items** permet
 - En mode conception de saisir du texte
 - En mode programmation d'utiliser **Add** et **Insert** pour ajouter du texte. `DomainUpDown1.Items.Add(leTexte)` –
`DomainUpDown1.Items.Insert(position, leTexte)`
 - Événement important **SelectedItemChanged** s'exécute lorsqu'un élément a été sélectionné

■ Autres types d'objets

- Autres listes : **DomainUpDown** - **NumericUpDown**
 - ✓ NumericUpDown se présente comme l'objet DomainUpDown mais permet de définir liste de numéro
 - Propriété ***Minimum*** permet de fixer la valeur minimale
 - Propriété ***Maximum*** permet de fixer la valeur maximale
 - Propriété ***Increment*** permet de définir le pas d'incrément
 - Propriété ***Value*** définir la valeur courante
 - Événement important ***SelectedItemChanged*** s'exécute lorsqu'un élément a été sélectionné

■ Autres types d'objets

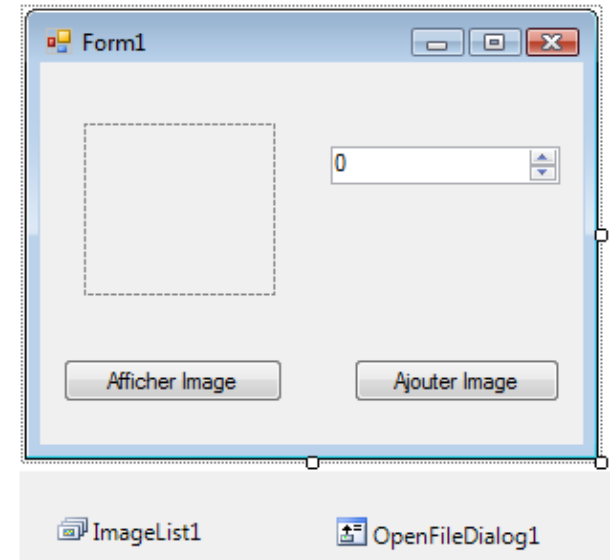
- Objets de type boîte de dialogue : **OpenFileDialog** – **SaveFileDialog**
 - ✓ Ils dérivent de la classe de base **CommonDialog**
 - ✓ Ils disposent des propriétés :
 - **Filter** : définit le contenu de la liste des types de fichiers
 - **InitialDirectory** : définit le répertoire par défaut
 - **Filename** : récupère le nom du fichier
 - ✓ Et de la méthode **ShowDialog()**; ouvre une boîte de dialogue pour choisir/écrire un nom de fichier et retourne la valeur :
 - **DialogResult.Ok** : si le bouton appuyé est Ouvrir/Enregistrer
 - **DialogResult.Cancel** : si le bouton appuyé est Annuler

■ Autres types d'objets

- L'objet de type boîte de dialogue : **SaveFileDialog**
 - ✓ disposent des propriétés :
 - **CreatePrompt** : à **True** message de confirmation si création de nouveau fichier
 - **OverWritePrompt** : à **True** affiche un message si le fichier existe déjà
 - **DefaultExt** : définit l'extension par défaut des noms du fichier
- Autres objets de type boîte de dialogue :
 - ✓ **ColorDialog** : pour le choix des couleurs
 - ✓ **FontDialog** : pour le choix des polices de caractères
 - ✓ **PrintDialog** : pour l'impression

■ Travaux pratiques N° 3

- Réaliser cette interface
 1. Écrire le code de sorte que quand on clique sur « Ajouter Image » on puisse choisir une image et l'insérer dans la liste d'images.
 2. Écrire du code de sorte que quand on clique sur « Afficher Image » l'image de rang donné par l'objet NumericUpDown est affichée dans le PictureBox



■ Autres types d'objets

- Objets de liste d'images : **ListView - TreeView**
 - ✓ ils disposent d'éléments image et texte
 - ✓ **ListView** dispose des propriétés suivantes :
 - **View** : définit le mode d'affichage
 - **LargeImageList – SmallImageList – StateImageList** : définit l'objet **ImageList** à associer au **ListView**
 - **Items** : définit la collection d'éléments; ses méthodes
 - **Add(leTexte, numImage)** : ajoute une image et un texte en dessous
 - **Remove(i)** : supprime l'élément de rang i
 - **MultiSelect** : à **True** permet la sélection multiple d'éléments
 - Un élément d'un **ListView** est désigné par la classe **ListViewItem**
 - **SelectedItems** : définit la collection d'éléments sélectionnés

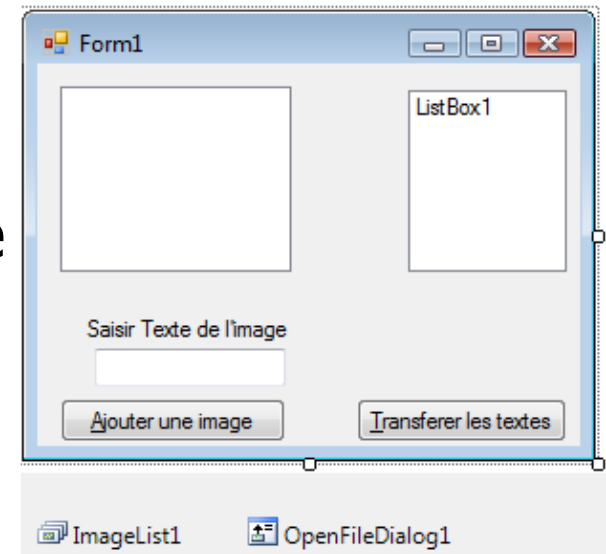
■ Autres types d'objets

- Objets de liste d'images : **ListView**
 - ✓ **ListView** dispose des propriétés suivantes :
 - **SelectedItems** : définit la collection d'éléments sélectionnés
 - **ListView1.SelectedItems[i].Text** : récupère le texte de l'élément sélectionné de rang i
 - **ListView1.SelectedItems[i].ImageIndex** : renvoie l'index de l'image dans **ImageList** associé de l'élément sélectionné de rang i
 - ✓ **ListView** dispose de l'événement **SelectedIndexChanged** qui s'exécute à chaque fois qu'un élément est sélectionné

Visual Studio

■ Travaux pratiques N° 4

- Réaliser cette interface
 - ✓ sur le click de « Ajouter une image » une fenêtre « Ouvrir » s'affiche on choisit une image, puis l'image et le texte saisis dans le textBox s'affichent dans le listView.
 - ✓ Quand on clique sur « Transférer les textes » les textes des éléments sélectionnés dans l'objet listView sont copiés dans l'objet ListBox.



■ Autres types d'objets

- Objets de liste d'images : **TreeView**

■ Autres types d'objets

- Le contrôle **RichTextBox**
 - ✓ Permet de faire du traitement de texte comme *WordPad*
 - ✓ Mise en forme du texte sélectionné :
 - ***RichTextBox1.SelectionColor = Color.Red*** applique la couleur rouge au texte sélectionné
 - ***RichTextBox1.SectionFont = New Font(police, taille, style)*** applique une police, une taille et un style au texte sélectionné
 - Police : chaîne de caractères "***Times New Roman***" par exemple
 - Style : ***FontStyle.Bold*** par exemple
 - ✓ Dispose des méthodes suivantes :
 - ***SaveFile(nomfichier, mode)*** enregistre le texte dans le fichier spécifié
 - mode peut être : ***RichTextBoxStreamType.{RichText | PlainText}***

■ Autres types d'objets

- Le contrôle **RichTextBox**
 - ✓ Dispose des méthodes suivantes :
 - ***LoadFile(nomfichier, mode)*** ouvre un fichier et remplit le RichTextBox
 - Les méthodes ***SaveFile()*** et ***LoadFile()*** sont utilisées conjointement avec ***SaveFileDialog*** et ***OpenFileDialog***
 - ***Find(TexteRecherché, PositionDébut, PositionFin, Option)*** recherche du texte et retourne l'index du premier caractère du texte cherché
 - Option peut prendre les valeurs : ***RichTextBoxFinds.MatchCase | NoHighlight | None | Reverse | WholeWord***
 - ***Copy – Cut – Paste*** : permet de copier, couper et coller le texte sélectionné

■ Autres types d'objets

- Le contrôle **RichTextBox**
 - ✓ Dispose des méthodes et propriétés suivantes :
 - On peut tester si certaines mises en forme ont été appliquées au texte sélectionné
 - ***SelectionFont.{UnderLine | StrickOut | Italic | Bold}*** retourne ***True*** si le texte est dans cette mise en forme
 - ***SelectionColor.<nomCouleur>*** retourne ***True*** si le texte sélectionné a la couleur spécifiée
 - On peut récupérer taille, nom de la police et couleur du texte sélectionné ***SelectionFont.Size – SelectionFont.Name – SelectionColor.toString()***
 - Les propriétés ***SelectionStart*** et ***SelectionLength*** donnent point de départ et nombre de caractères du texte sélectionné
 - Les méthodes ***Undo***, ***Redo*** disponibles pour les dernières modifications

■ Autres types d'objets

- Objets de type menu : **MenuStrip** - **ContextMenuStrip**
 - ✓ On saisit les éléments de menu dans la zone « Tapez ici »; une fois un éléments de menu saisi, une zone « Tapez ici » apparaît à droite pour saisir un autre élément de menu et une zone « Tapez ici » apparaît en bas pour saisir les éléments de sous-menu
 - ✓ Pour associer un menu contextuel (**ContextMenuStrip**) à un contrôle, on affecte à la propriété **ContextMenuStrip** du contrôle le nom de l'objet **ContextMenuStrip**
- L'objet barre d'outils : **ToolStrip** permet facilement de définir une barre d'outils avec les éléments standards

■ Application MDI

- MDI (Multiple Document Interface) comprend :
 - ✓ Un formulaire unique parent conteneur, la propriété ***IsMDIContainer*** de ce formulaire est à ***True***
 - ✓ Des formulaires enfants, leur propriété ***MDIParent*** a pour valeur le nom du formulaire parent (conteneur MDI)
- Affichage/fermeture d'un formulaire
 - ✓ ***Application.Run(New nomFormulaire);***
 - ✓ Ou bien on suit les étapes suivantes :
 - On crée une instance du formulaire
 - On affiche par : ***nom_instanceFormulaire.Show()***
 - ✓ On ferme par sa méthode ***Close()*** ou ***Application.Exit()***

■ **Projet**

- Réaliser un petit éditeur de textes à l'aide d'un RichTextBox et des objets OpenFileDialog et SaveFileDialog. On devrait pouvoir :
 - ✓ ajouter un menu, une barre d'outils
 - ✓ faire toutes les mises en forme
 - ✓ rechercher et remplacer du texte
 - ✓ ouvrir un fichier existant et enregistrer/enregistrer sous un autre nom un fichier
 - ✓ Optionnellement avoir plusieurs fenêtres